

M.Abdullah.Mahmood

BCS-3B

22F-3289

To: Sir Abdul Qadeer

Data Structures LAB – 02

Task 1:

Code:

```
#include<iostream>
using namespace std;

class Node {
public:
    Node() : next(nullptr) {};

    Node(int key) {
        this->key = key;
        next = NULL;
    }

    int key;
    Node* next;
};

class LinkedList {
public:
    Node* head = new Node();

    LinkedList():head(nullptr) {};

    void insertAtPos(int value, int pos) {

        if (pos == 1) {
            Node* temp = new Node(value);
            temp->next = head;
            head = temp;
        }
    }
};
```

```

    }
    else {
        Node* temp = new Node(value);
        Node* current = head;

        while (--pos > 1)
            current = current->next;

        temp->next = current->next;
        current->next = temp;
    }
}

void searchElement(int value, Node* head) {
    if (head == NULL) {
        cout << "no element in list \n";
        return;
    }
    else {
        if (value == head->key) {
            cout << "value found !! \n";
            return;
        }
        else {
            searchElement(value, head->next);
        }
    }
}

~LinkedList(){
    delete head;
}

};

int main()
{
    LinkedList L;
    L.insertAtPos(2,1);
    L.insertAtPos(3,2);
    L.insertAtPos(4,3);
    L.insertAtPos(5,4);

    L.searchElement(3,L.head);
    return 0;
}

```

Output:

```
PS C:\Users\USER\OneDrive\Desktop\lab
if ($?) { .\task1 }
value found !!
```

Task 2 :

CODE:

```
#include<iostream>
using namespace std;

class Node {
public:
    Node() : next(nullptr) {};

    Node(int key) {
        this->key = key;
        next = NULL;
    }

    int key;
    Node* next;
};

class LinkedList {
public:
    Node* head = new Node();

    LinkedList():head(nullptr) {};

    void insertAtPos(int value, int pos) {

        if (pos == 1) {
            Node* temp = new Node(value);
            temp->next = head;
            head = temp;
        }
        else {
            Node* temp = new Node(value);
            Node* current = head;
```

```

        while (--pos > 1)
            current = current->next;

        temp->next = current->next;
        current->next = temp;
    }
}

void secondMax(){
    Node* current = new Node();
    // Node* next = new Node();
    current = head;

    int max1 = head->key ;
    int max2 = -1;

    for(int i = 1 ; current != NULL ; ++i){
        if(current->key > max1){
            max2 = max1;
            max1 = current->key;
        }
        else if(current->key < max1 && current->key > max2 )
            max2 = current->key;
        current = current -> next;
    }

    cout<<"2nd max is : "<<max2<<endl;
}

~LinkedList(){
    delete head;
}
};

int main()
{
    LinkedList L;
    L.insertAtPos(2,1);
    L.insertAtPos(9,2);
    L.insertAtPos(4,3);
    L.insertAtPos(5,4);

    L.secondMax();
}

```

```
    return 0;
}
```

Output:

```
● PS C:\Users\USER\OneDrive\Desktop\lab 3> cd "
● if ($?) { .\task2 }
  2nd max is : 5
○ PS C:\Users\USER\OneDrive\Desktop\lab 3> █
```

Task 3:

Code:

```
#include<iostream>
using namespace std;

class Node {
public:
    Node() : next(nullptr) {};

    Node(int data) {
        this->data = data;
        next = NULL;
    }

    int data;
    Node* next;
};

class LinkedList {
public:
```

```

Node* head ;
static int size;
LinkedList(): head(nullptr) {};

void insertAtPos(int value, int pos) {
    ++size;
    if (pos == 1) {
        Node* temp = new Node(value);
        temp->next = head;
        head = temp;
    }
    else {
        Node* temp = new Node(value);
        Node* current = head;

        while (--pos > 1)
            current = current->next;

        temp->next = current->next;
        current->next = temp;
    }
}

```

```

bool sizeCheck(){

    //if even then at start

    if(size/2 == 0){
        insertAtPos(5,1);
        return true;
    }

    //if odd then on size/2

    else{
        insertAtPos(5,size/2);
        return false;
    }
}

```

```

void print() {
    Node* current = head;

    while (current) {

```

```

        cout << current->data << " ";
        current = current->next;
    }
}

~LinkedList() {}
};

int LinkedList::size = 0;

int main()
{
    LinkedList L;
    L.insertAtPos(1,1);
    L.insertAtPos(2,2);
    L.insertAtPos(3,3);
    L.insertAtPos(4,4);
    L.insertAtPos(6,5);

    if(L.sizeCheck()){
        L.print();
        cout<<"\n its even \n";
    }
    else{
        L.print();
        cout<<"\n its odd \n";
    }

    return 0;
}

```

Output:

```

PS C:\Users\USER\OneDrive\Desktop\lab
3 }
● 1 5 2 3 4 6
  its odd

```

Task 4:

Code:

```
#include <iostream>
using namespace std;

class Node {
public:
    string songName;
    string artist;
    Node* next;

    Node() : next(nullptr) {}
};

class LinkedList {
public:
    Node* head;

    LinkedList() : head(nullptr) {}

    void printPlaylist() {
        Node* current = head;

        if (current == nullptr) {
            cout << "Playlist is empty." << endl;
            return;
        }

        cout << "Playlist songs:" << endl;

        while (current) {
            cout << "Song: " << current->songName << " by " << current->artist <<
endl;
            current = current->next;
        }
        return;
    }

    void addSong(int position) {
        if (position < 0) {
            cout << "Invalid position. Song not added." << endl;
            return;
        }
    }
};
```

```

    }

    Node* newNode = new Node();
    cout << "Enter song name:\n";
    cin >> newNode->songName;
    cout << "Enter artist:\n";
    cin >> newNode->artist;

    if (position == 0) {
        newNode->next = head;
        head = newNode;
    } else {
        Node* current = head;
        int currentPosition = 0;

        while (current && currentPosition < position - 1) {
            current = current->next;
            currentPosition++;
        }

        if (current == nullptr) {
            cout << "Invalid position. Song not added." << endl;
            delete newNode;
            return;
        }

        newNode->next = current->next;
        current->next = newNode;
    }

    cout << "Song added successfully!" << endl;
}

void removeSong(int position) {
    if (position < 0) {
        cout << "Invalid position. Song not removed." << endl;
        return;
    }

    if (head == nullptr) {
        cout << "Playlist is empty. Song not removed." << endl;
        return;
    }

    if (position == 0) {

```

```

        Node* temp = head;
        head = head->next;
        delete temp;
        cout << "Song removed successfully!" << endl;
        return;
    }

    Node* current = head;
    int currentPosition = 0;

    //reaching pos - 1
    while (current && currentPosition < position - 1) {
        current = current->next;
        currentPosition++;
    }

    //if pos-1 is pointing to NULL means there is no song ahead to download
    if (current == nullptr || current->next == nullptr) {
        cout << "Invalid position. Song not removed." << endl;
        return;
    }

    Node* temp = current->next;
    current->next = temp->next;
    delete temp;
    cout << "Song removed successfully!" << endl;
}

~LinkedList() {
    Node* current = head;
    while (current) {
        Node* temp = current;
        current = current->next;
        delete temp;
    }
}

};

int main() {
    LinkedList L;
    L.addSong(0);
    L.addSong(1);
    L.addSong(2);
    L.printPlaylist();
}

```

```
L.removeSong(1);  
L.printPlaylist();  
  
return 0;  
}
```

Output:

```
PS C:\Users\USER\OneDrive\Desktop\lab 3> cd "c:\Users\USER\OneDrive\Desktop\lab 3" & if ($?) { .\task4 }  
Enter song name:  
metYou  
Enter artist:  
me  
Song added successfully!  
Enter song name:  
YOU  
Enter artist:  
u  
Song added successfully!  
Enter song name:  
I  
Enter artist:  
0
```

```
Song added successfully!
Enter song name:
I
Enter artist:
O
Song added successfully!
Playlist songs:
Song: metYou by me
Song: YOU by u
Song: I by O
Song removed successfully!
Playlist songs:
Song: metYou by me
Song: I by O
PS C:\Users\USER\OneDrive\Desktop\lab 3>
```

Task 5:

Code:

```
#include<iostream>
#include<string>
using namespace std;

class Node {
public:
```

```

    string line;
    Node* next;

    Node() : next(nullptr) {};
    Node(string data) : line(data), next(nullptr) {};

    ~Node() {};
};

class LinkedList {
public:
    Node* head;
    static int num;
    LinkedList() : head(nullptr) {};

    void insertLine(int pos, string data) {

        ++num;

        if (pos > num) {
            cout << "invalid position !! \n";
            exit(0);
        }

        if (pos == 0) {
            Node* current = new Node(data);
            current->next = head;
            head = current;
        }
        else {
            Node* newNode = new Node(data);
            Node* cur = head;

            while (--pos > 1 && cur != NULL) {
                cur = cur->next;
            }

            newNode->next = cur->next;
            cur->next = newNode;

        }

        return;
    }
};

```

```

void deleteLine(int pos) {
    if (pos > num) {
        cout << "invalid input \n";
        exit(0);
    }

    Node* curr = head;
    Node* prev = NULL;
    if (pos == 0) {
        curr = curr->next;
    }
    else {
        while (--pos >= 1) {
            prev = curr;
            curr = curr->next;
        }
        prev->next = curr->next;

        Node* temp = new Node();
        temp = curr;
        delete temp;
    }
    return;
}

void editLine(int pos) {
    if (pos > num) {
        cout << "invalid line num\n";
        exit(0);
    }

    Node* curr = head;
    while (--pos >= 1) {
        curr = curr->next;
    }

    cout << "enter new data \n";
    getline(cin, curr->line);

    return;
}

void getline(int pos){
    if (pos > num) {

```

```

        cout << "invalid line num\n";
        exit(0);
    }

    Node* curr = head;
    while (--pos >= 1) {
        curr = curr->next;
    }

    cout << "data :  "<<curr->line<<endl;

    return;
}

void displayDoc() {
    Node* current = head;
    while (current) {
        cout << current->line << " ";
        current = current->next;
    }
    return;
}

~LinkedList() {}
};

int LinkedList::num = 0;
int main()
{
    LinkedList L;
    L.insertLine(0, "abdullah");
    L.insertLine(1, "ali");
    L.insertLine(2, "zamil");
    L.insertLine(1, "bilal");
    L.insertLine(0, "taha");
    L.insertLine(4, "t");

    L.displayDoc();
    cout << "\n";

    //    L.deleteLine(6);
    //    L.displayDoc();
    L.editLine(6);
    cout << "\n";
    L.displayDoc();

```

```
cout << endl;  
L.getLine(5);  
  
return 0;  
}
```
